

Name:-Maithili Mahajan

Section(Batch):-B2-B1

Roll No:-12

✓ Practical No:-8 : Part C Classification Models

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

✓ 1. Dataset

```
df=pd.read_csv("breast_cancer.csv")
# First 5 rows
print("\n----- FIRST 5 ROWS -----")
print(df.head())
```

```
----- FIRST 5 ROWS -----
   id  diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0   842302      M        17.99         10.38          122.80       1001.0
1   842517      M        20.57         17.77          132.90       1326.0
2  84300903      M        19.69         21.25          130.00       1203.0
3  84348301      M        11.42         20.38           77.58        386.1
4  84358402      M        20.29         14.34          135.10       1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0         0.11840         0.27760         0.3001         0.14710
1         0.08474         0.07864         0.0869         0.07017
2         0.10960         0.15990         0.1974         0.12790
3         0.14250         0.28390         0.2414         0.10520
4         0.10030         0.13280         0.1980         0.10430

   ... texture_worst  perimeter_worst  area_worst  smoothness_worst  \
0   ...          17.33           184.60       2019.0         0.1622
1   ...          23.41           158.80       1956.0         0.1238
2   ...          25.53           152.50       1709.0         0.1444
3   ...          26.50           98.87        567.7         0.2098
4   ...          16.67           152.20       1575.0         0.1374

   compactness_worst  concavity_worst  concave points_worst  symmetry_worst  \
0         0.6656         0.7119         0.2654         0.4601
1         0.1866         0.2416         0.1860         0.2750
2         0.4245         0.4504         0.2430         0.3613
3         0.8663         0.6869         0.2575         0.6638
4         0.2050         0.4000         0.1625         0.2364

   fractal_dimension_worst  Unnamed: 32
0         0.11890              NaN
```

```

1          0.08902      NaN
2          0.08758      NaN
3          0.17300      NaN
4          0.07678      NaN

```

```
[5 rows x 33 columns]
```

```

# Last 5 rows
df.tail(5)

```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000

```
5 rows x 33 columns
```

```

# Number of rows and columns
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

```

```

Rows: 569
Columns: 33

```

```

# Column names
df.columns.tolist()

```

```

['id',
 'diagnosis',
 'radius_mean',
 'texture_mean',
 'perimeter_mean',
 'area_mean',
 'smoothness_mean',
 'compactness_mean',
 'concavity_mean',
 'concave points_mean',
 'symmetry_mean',
 'fractal_dimension_mean',
 'radius_se',
 'texture_se',
 'perimeter_se',
 'area_se',
 'smoothness_se',
 'compactness_se',
 'concavity_se',
 'concave points_se',
 'symmetry_se',
 'fractal_dimension_se',
 'radius_worst',
 'texture_worst',
 'perimeter_worst',
 'area_worst',
 'smoothness_worst',
 'compactness_worst',
 'concavity_worst',
 'concave points_worst',
 'symmetry_worst',
 'fractal_dimension_worst',
 'Unnamed: 32']

```

```

# Data types
df.dtypes

```

```

      0
      id      int64
      diagnosis object
      radius_mean float64
      texture_mean float64
      perimeter_mean float64
      area_mean float64
      smoothness_mean float64
      compactness_mean float64
      concavity_mean float64
      concave points_mean float64
      symmetry_mean float64
      fractal_dimension_mean float64
      radius_se float64
      texture_se float64
      perimeter_se float64
      area_se float64
      smoothness_se float64
      compactness_se float64
      concavity_se float64
      concave points_se float64
      symmetry_se float64
      fractal_dimension_se float64
      radius_worst float64
      texture_worst float64
      perimeter_worst float64
      area_worst float64
      smoothness_worst float64
      compactness_worst float64
      concavity_worst float64
      concave points_worst float64
      symmetry_worst float64
      fractal_dimension_worst float64
      Unnamed: 32 float64

```

dtype: object

```
# Dataset information
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    569 non-null   int64
1   diagnosis             569 non-null   object
2   radius_mean          569 non-null   float64
3   texture_mean         569 non-null   float64
4   perimeter_mean       569 non-null   float64
5   area_mean            569 non-null   float64
6   smoothness_mean      569 non-null   float64
7   compactness_mean     569 non-null   float64
8   concavity_mean       569 non-null   float64
9   concave points_mean  569 non-null   float64

```

```
10 symmetry_mean      569 non-null    float64
11 fractal_dimension_mean  569 non-null    float64
12 radius_se          569 non-null    float64
13 texture_se         569 non-null    float64
14 perimeter_se       569 non-null    float64
15 area_se            569 non-null    float64
16 smoothness_se     569 non-null    float64
17 compactness_se    569 non-null    float64
18 concavity_se      569 non-null    float64
19 concave points_se  569 non-null    float64
20 symmetry_se       569 non-null    float64
21 fractal_dimension_se  569 non-null    float64
22 radius_worst      569 non-null    float64
23 texture_worst     569 non-null    float64
24 perimeter_worst   569 non-null    float64
25 area_worst        569 non-null    float64
26 smoothness_worst  569 non-null    float64
27 compactness_worst  569 non-null    float64
28 concavity_worst   569 non-null    float64
29 concave points_worst  569 non-null    float64
30 symmetry_worst    569 non-null    float64
31 fractal_dimension_worst  569 non-null    float64
32 Unnamed: 32       0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

```
# Missing values
df.isnull().sum()
```

	0
id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0
texture_worst	0
perimeter_worst	0
area_worst	0
smoothness_worst	0
compactness_worst	0
concavity_worst	0
concave points_worst	0
symmetry_worst	0
fractal_dimension_worst	0
Unnamed: 32	569

dtype: int64

```
#Duplicate records
print("Number of duplicates:", df.duplicated().sum())
```

Number of duplicates: 0

2. Data Pre-processing

```
# Keep only required columns
df = df.drop(columns=["id", "Unnamed: 32"])
print("\n----- DATASET AFTER REMOVING UNNECESSARY COLUMNS -----")
print(df.head())
print("\nNew shape:", df.shape)
print("\nColumns used for classification:")
print(df.columns.tolist())
```

----- DATASET AFTER REMOVING UNNECESSARY COLUMNS -----

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
0	M	17.99	10.38	122.80	1001.0	
1	M	20.57	17.77	132.90	1326.0	
2	M	19.69	21.25	130.00	1203.0	
3	M	11.42	20.38	77.58	386.1	
4	M	20.29	14.34	135.10	1297.0	

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	\
0	0.11840	0.27760	0.3001	0.14710	
1	0.08474	0.07864	0.0869	0.07017	
2	0.10960	0.15990	0.1974	0.12790	
3	0.14250	0.28390	0.2414	0.10520	
4	0.10030	0.13280	0.1980	0.10430	

	symmetry_mean	...	radius_worst	texture_worst	perimeter_worst	\
0	0.2419	...	25.38	17.33	184.60	
1	0.1812	...	24.99	23.41	158.80	
2	0.2069	...	23.57	25.53	152.50	
3	0.2597	...	14.91	26.50	98.87	
4	0.1809	...	22.54	16.67	152.20	

	area_worst	smoothness_worst	compactness_worst	concavity_worst	\
0	2019.0	0.1622	0.6656	0.7119	
1	1956.0	0.1238	0.1866	0.2416	
2	1709.0	0.1444	0.4245	0.4504	
3	567.7	0.2098	0.8663	0.6869	
4	1575.0	0.1374	0.2050	0.4000	

	concave points_worst	symmetry_worst	fractal_dimension_worst
0	0.2654	0.4601	0.11890
1	0.1860	0.2750	0.08902
2	0.2430	0.3613	0.08758
3	0.2575	0.6638	0.17300
4	0.1625	0.2364	0.07678

[5 rows x 31 columns]

New shape: (569, 31)

Columns used for classification:

['diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_me

```
# Remove Duplicates
duplicate_count = df.duplicated().sum()
print("\nDuplicate records:", duplicate_count)
if duplicate_count > 0:
    df = df.drop_duplicates()
    print("Duplicates removed.")

print("Final shape:", df.shape)
```

Duplicate records: 0
Final shape: (569, 31)

```
# Handle missing Values
print("\n MISSING VALUES AFTER CLEANING")
print(df.isnull().sum())

# Fill numerical missing values with median if any
numerical_columns = df.select_dtypes(include=np.number).columns

for col in numerical_columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].median())

# Fill categorical missing values with mode if any
categorical_columns = df.select_dtypes(include="object").columns

for col in categorical_columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].mode()[0])

print("\nMissing values handled.")
```

MISSING VALUES AFTER CLEANING

```

diagnosis          0
radius_mean        0
texture_mean        0
perimeter_mean     0
area_mean          0
smoothness_mean    0
compactness_mean   0
concavity_mean     0
concave points_mean 0
symmetry_mean      0
fractal_dimension_mean 0
radius_se          0
texture_se         0
perimeter_se       0
area_se            0
smoothness_se      0
compactness_se     0
concavity_se       0
concave points_se  0
symmetry_se        0
fractal_dimension_se 0
radius_worst       0
texture_worst      0
perimeter_worst    0
area_worst         0
smoothness_worst   0
compactness_worst  0
concavity_worst    0
concave points_worst 0
symmetry_worst     0
fractal_dimension_worst 0
dtype: int64

```

Missing values handled.

3. Exploratory Data Analysis (EDA)

```

# Numerical and categorical variables
numerical_columns = df.select_dtypes(include=np.number).columns.tolist()
categorical_columns = df.select_dtypes(include="object").columns.tolist()

print("\nNumerical Variables ")
print(numerical_columns)

print("\nCategorical Variables ")
print(categorical_columns)

```

```

Numerical Variables
['radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean', 'concave
Categorical Variables
['diagnosis']

```

```

# Convert categorical target into numerical form
print("\nOriginal diagnosis values:")
print(df["diagnosis"].value_counts())
df["diagnosis"] = df["diagnosis"].map({
    "B": 0,
    "M": 1
})

print("\nEncoded diagnosis:")
print(df["diagnosis"].value_counts())

print("\n0 = Benign")
print("1 = Malignant")

```

```

Original diagnosis values:
diagnosis
B      357
M      212
Name: count, dtype: int64

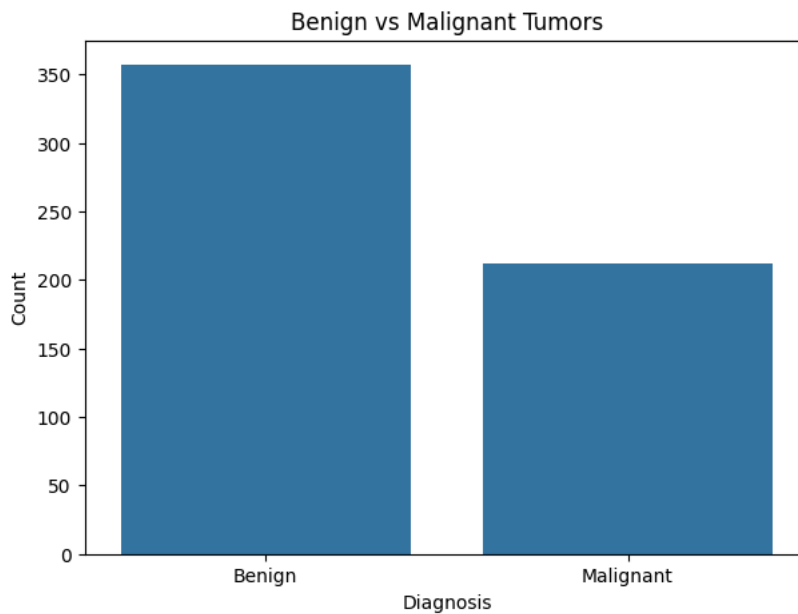
Encoded diagnosis:
diagnosis

```

```
0    357
1    212
Name: count, dtype: int64

0 = Benign
1 = Malignant
```

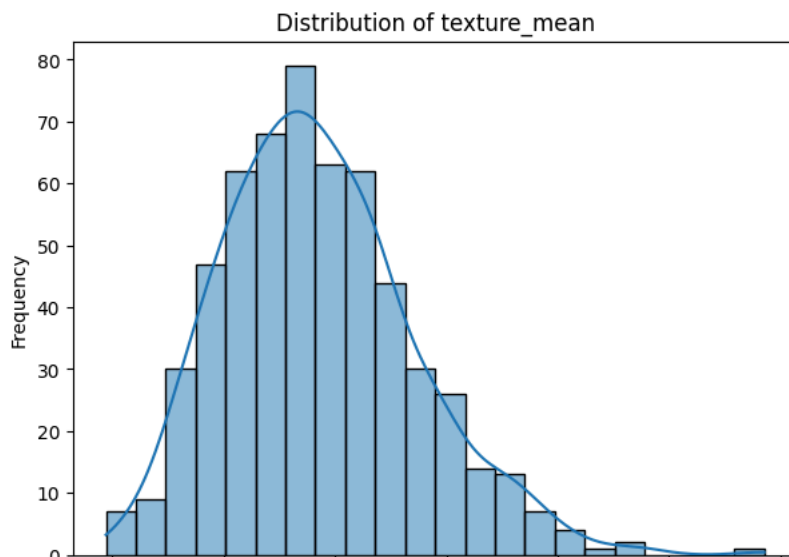
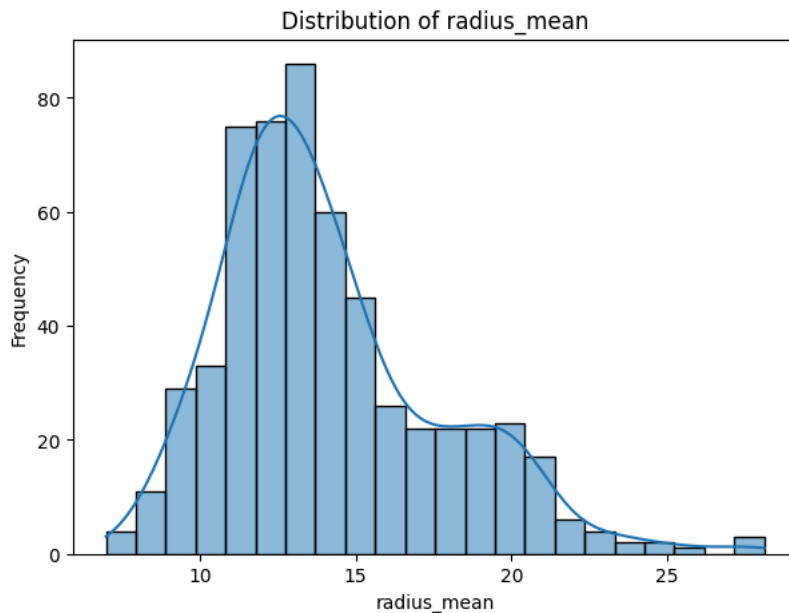
```
# Distribution plots
plt.figure(figsize=(7, 5))
sns.countplot( data=df,x="diagnosis")
plt.title("Benign vs Malignant Tumors")
plt.xlabel("Diagnosis")
plt.ylabel("Count")
plt.xticks([0, 1],["Benign", "Malignant"])
plt.show()
```



```
# Selected important features
selected_features = [
    "radius_mean",
    "texture_mean",
    "perimeter_mean",
    "area_mean",
    "smoothness_mean"
]

for column in selected_features:

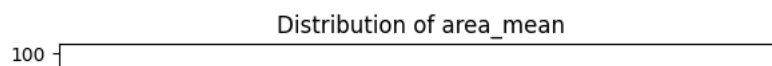
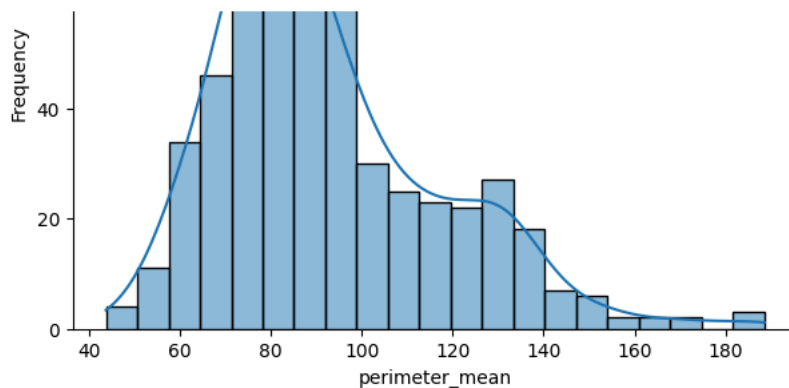
    plt.figure(figsize=(7, 5))
    sns.histplot(data=df,x=column,kde=True)
    plt.title("Distribution of " + column)
    plt.xlabel(column)
    plt.ylabel("Frequency")
    plt.show()
```

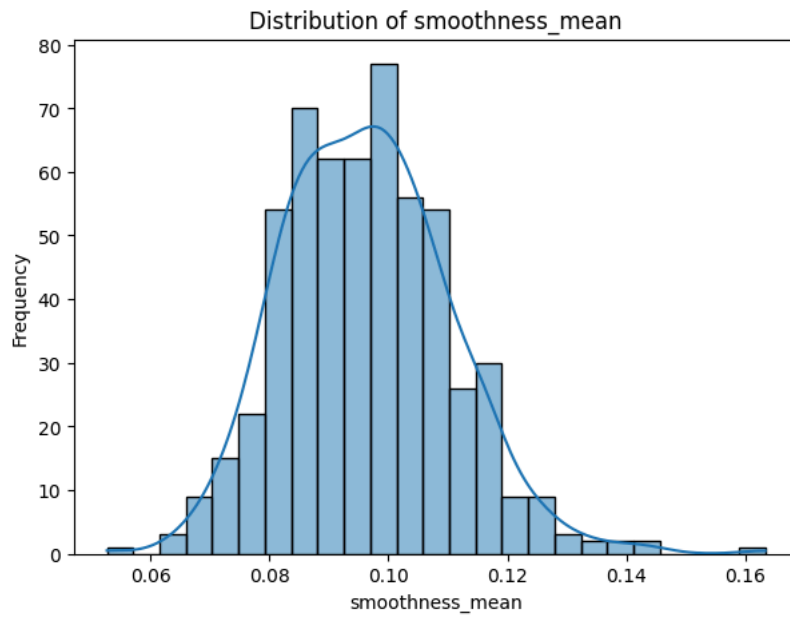
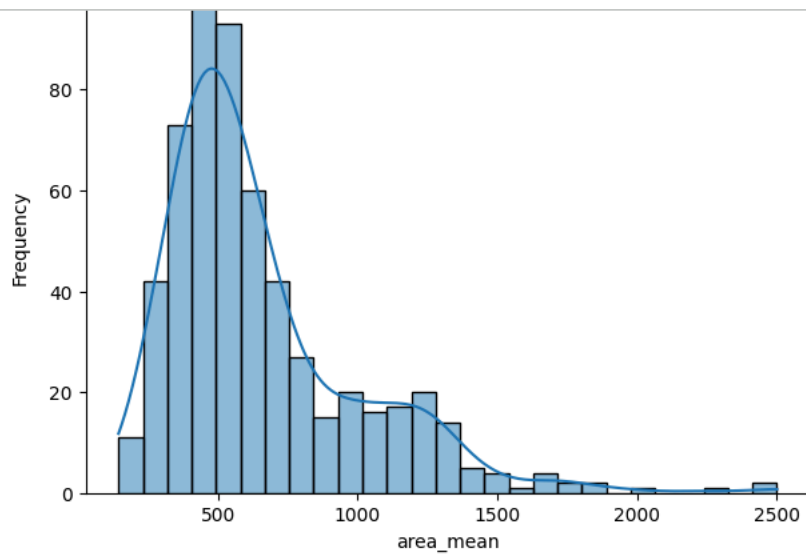



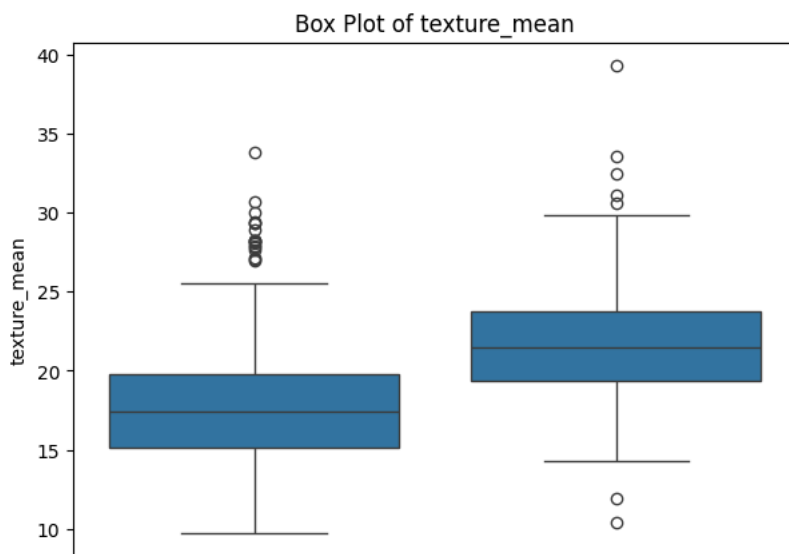
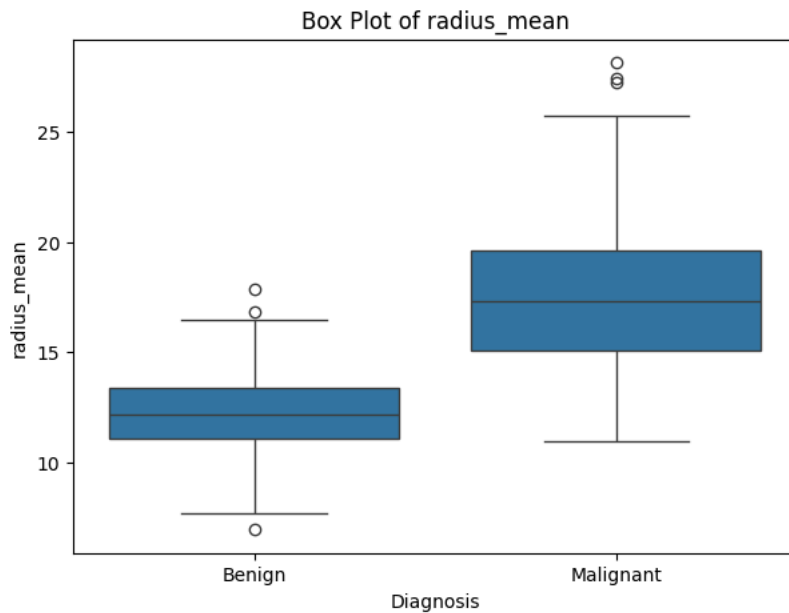
```
# BOXPLOTS
for column in selected_features:

    plt.figure(figsize=(7, 5))
    sns.boxplot(data=df, x="diagnosis", y=column)
    plt.title("Box Plot of " + column)
    plt.xlabel("Diagnosis")
    plt.ylabel(column)

    plt.xticks([0, 1], ["Benign", "Malignant"])
    plt.show()
```







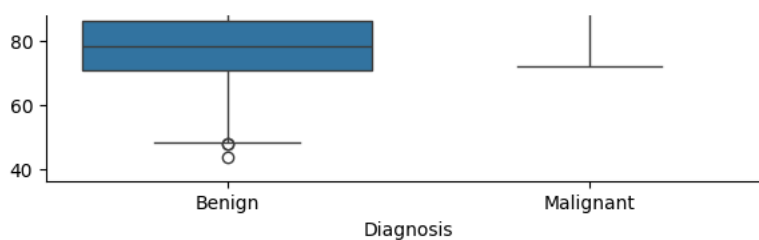
```
# Scatter Plot
plt.figure(figsize=(8, 6))

sns.scatterplot(data=df, x="radius_mean", y="area_mean", hue="diagnosis")
plt.title("Radius Mean vs Area Mean")
plt.xlabel("Radius Mean")
plt.ylabel("Area Mean")

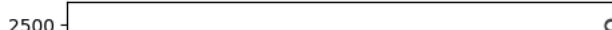
plt.show()

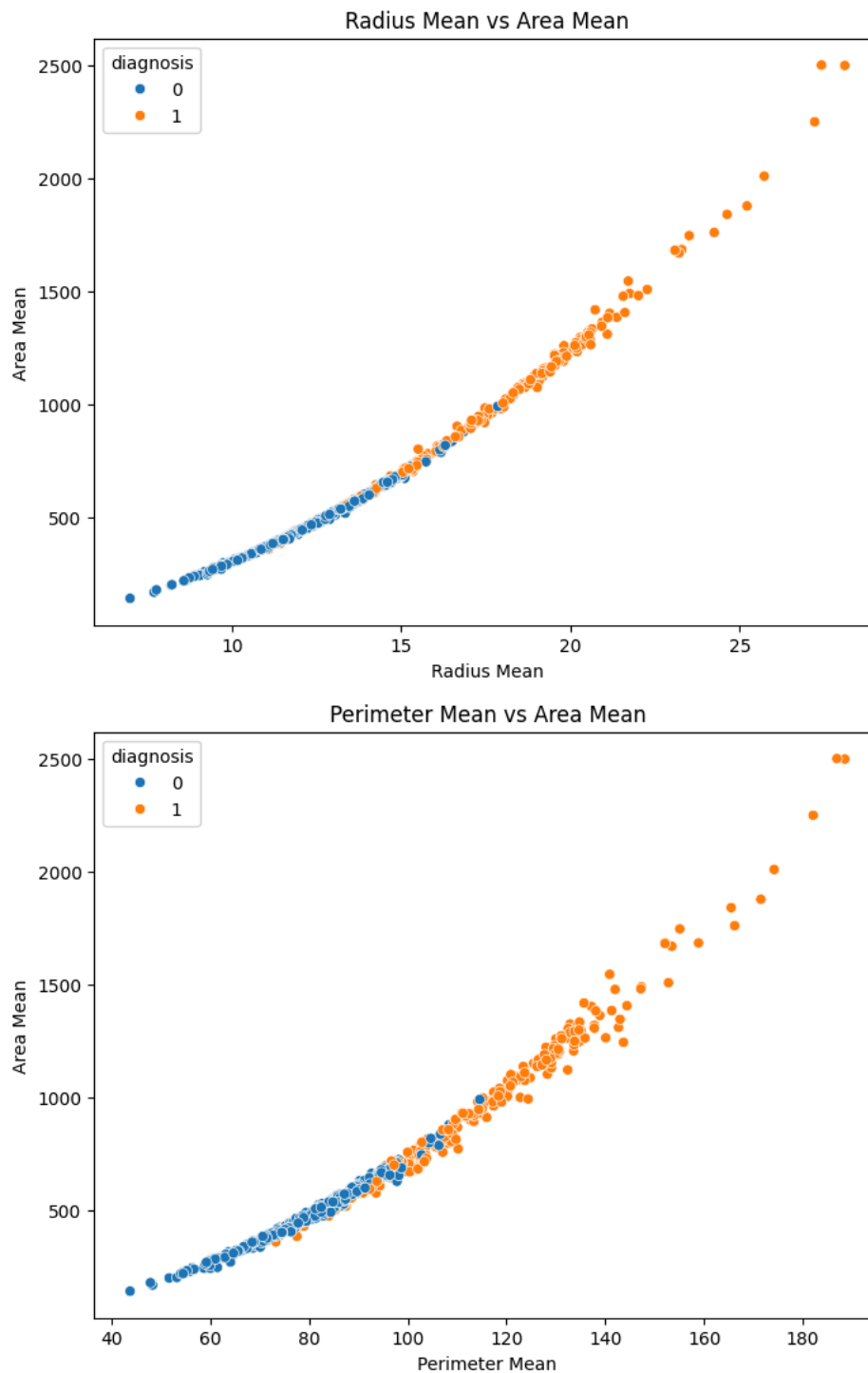
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x="perimeter_mean", y="area_mean", hue="diagnosis")
plt.title("Perimeter Mean vs Area Mean")
plt.xlabel("Perimeter Mean")
plt.ylabel("Area Mean")

plt.show()
```



Box Plot of area_mean





4. Correlation Analysis

```
correlation_matrix = df.corr()

print("\nCorrelation Matrix")
print(correlation_matrix)

plt.figure(figsize=(18, 14))
sns.heatmap(correlation_matrix, cmap="coolwarm", annot=False)

plt.title("Correlation Heatmap")
plt.show()
```


CoRelation Matrix

	diagnosis	radius_mean	texture_mean	perimeter_mean	\
diagnosis	1.000000	0.730029	0.415185	0.742636	
radius_mean	0.730029	1.000000	0.323782	0.997855	
texture_mean	0.415185	0.323782	1.000000	0.329533	
perimeter_mean	0.742636	0.997855	0.329533	1.000000	
area_mean	0.708984	0.987357	0.321086	0.986507	
smoothness_mean	0.358560	0.170581	-0.023389	0.207278	
compactness_mean	0.596534	0.506124	0.236702	0.556936	
concavity_mean	0.696360	0.676764	0.302418	0.716136	
concave points_mean	0.776614	0.822529	0.293464	0.850977	
symmetry_mean	0.330499	0.147741	0.071401	0.183027	
fractal_dimension_mean	-0.012838	-0.311631	-0.076437	-0.261477	
radius_se	0.567134	0.679090	0.275869	0.691765	
texture_se	-0.008303	-0.097317	0.386358	-0.086761	
perimeter_se	0.556141	0.674172	0.281673	0.693135	
area_se	0.548236	0.735864	0.259845	0.744983	
smoothness_se	-0.067016	-0.222600	0.006614	-0.202694	
compactness_se	0.292999	0.206000	0.191975	0.250744	
concavity_se	0.253730	0.194204	0.143293	0.228082	
concave points_se	0.408042	0.376169	0.163851	0.407217	
symmetry_se	-0.006522	-0.104321	0.009127	-0.081629	
fractal_dimension_se	0.077972	-0.042641	0.054458	-0.005523	
radius_worst	0.776454	0.969539	0.352573	0.969476	
texture_worst	0.456903	0.297008	0.912045	0.303038	
perimeter_worst	0.782914	0.965137	0.358040	0.970387	
area_worst	0.733825	0.941082	0.343546	0.941550	
smoothness_worst	0.421465	0.119616	0.077503	0.150549	
compactness_worst	0.590998	0.413463	0.277830	0.455774	
concavity_worst	0.659610	0.526911	0.301025	0.563879	
concave points_worst	0.793566	0.744214	0.295316	0.771241	
symmetry_worst	0.416294	0.163953	0.105008	0.189115	
fractal_dimension_worst	0.323872	0.007066	0.119205	0.051019	

	area_mean	smoothness_mean	compactness_mean	\
diagnosis	0.708984	0.358560	0.596534	
radius_mean	0.987357	0.170581	0.506124	
texture_mean	0.321086	-0.023389	0.236702	
perimeter_mean	0.986507	0.207278	0.556936	
area_mean	1.000000	0.177028	0.498502	
smoothness_mean	0.177028	1.000000	0.659123	
compactness_mean	0.498502	0.659123	1.000000	
concavity_mean	0.685983	0.521984	0.883121	
concave points_mean	0.823269	0.553695	0.831135	
symmetry_mean	0.151293	0.557775	0.602641	
fractal_dimension_mean	-0.283110	0.584792	0.565369	
radius_se	0.732562	0.301467	0.497473	
texture_se	-0.066280	0.068406	0.046205	
perimeter_se	0.726628	0.296092	0.548905	
area_se	0.800086	0.246552	0.455653	
smoothness_se	-0.166777	0.332375	0.135299	
compactness_se	0.212583	0.318943	0.738722	
concavity_se	0.207660	0.248396	0.570517	
concave points_se	0.372320	0.380676	0.642262	
symmetry_se	-0.072497	0.200774	0.229977	
fractal_dimension_se	-0.019887	0.283607	0.507318	
radius_worst	0.962746	0.213120	0.535315	
texture_worst	0.287489	0.036072	0.248133	
perimeter_worst	0.959120	0.238853	0.590210	
area_worst	0.959213	0.206718	0.509604	
smoothness_worst	0.123523	0.805324	0.565541	
compactness_worst	0.390410	0.472468	0.865809	
concavity_worst	0.512606	0.434926	0.816275	
concave points_worst	0.722017	0.503053	0.815573	
symmetry_worst	0.143570	0.394309	0.510223	
fractal_dimension_worst	0.003738	0.499316	0.687382	

	concavity_mean	concave points_mean	symmetry_mean	\
diagnosis	0.696360	0.776614	0.330499	
radius_mean	0.676764	0.822529	0.147741	
texture_mean	0.302418	0.293464	0.071401	
perimeter_mean	0.716136	0.850977	0.183027	
area_mean	0.685983	0.823269	0.151293	
smoothness_mean	0.521984	0.553695	0.557775	
compactness_mean	0.883121	0.831135	0.602641	
concavity_mean	1.000000	0.921391	0.500667	
concave points_mean	0.921391	1.000000	0.462497	
symmetry_mean	0.500667	0.462497	1.000000	
fractal_dimension_mean	0.336783	0.166917	0.479921	
radius_se	0.631925	0.698050	0.303379	
texture_se	0.076218	0.021480	0.128053	
perimeter_se	0.660391	0.710650	0.313893	
area_se	0.617427	0.690299	0.223970	

smoothness_se	0.098564	0.027653	0.181321
compactness_se	0.670279	0.490424	0.421659
concavity_se	0.691270	0.439167	0.342627
concave_points_se	0.683260	0.615634	0.393298
symmetry_se	0.178009	0.095351	0.449137
fractal_dimension_se	0.449301	0.257584	0.331786
radius_worst	0.688236	0.830318	0.185728
texture_worst	0.299879	0.292752	0.090651
perimeter_worst	0.729565	0.855923	0.219169
area_worst	0.675987	0.809630	0.177193
smoothness_worst	0.448822	0.452753	0.426675
compactness_worst	0.754968	0.667454	0.473200
concavity_worst	0.884103	0.752399	0.433721
concave_points_worst	0.861323	0.910155	0.430297
symmetry_worst	0.409464	0.375744	0.699826
fractal_dimension_worst	0.514930	0.368661	0.438413

	...	radius_worst	texture_worst	perimeter_worst	\
diagnosis	...	0.776454	0.456903	0.782914	
radius_mean	...	0.969539	0.297008	0.965137	
texture_mean	...	0.352573	0.912045	0.358040	
perimeter_mean	...	0.969476	0.303038	0.970387	
area_mean	...	0.962746	0.287489	0.959120	
smoothness_mean	...	0.213120	0.036072	0.238853	
compactness_mean	...	0.535315	0.248133	0.590210	
concavity_mean	...	0.688236	0.299879	0.729565	
concave_points_mean	...	0.830318	0.292752	0.855923	
symmetry_mean	...	0.185728	0.090651	0.219169	
fractal_dimension_mean	...	-0.253691	-0.051269	-0.205151	
radius_se	...	0.715065	0.194799	0.719684	
texture_se	...	-0.111690	0.409003	-0.102242	
perimeter_se	...	0.697201	0.200371	0.721031	
area_se	...	0.757373	0.196497	0.761213	
smoothness_se	...	-0.230691	-0.074743	-0.217304	
compactness_se	...	0.204607	0.143003	0.260516	
concavity_se	...	0.186904	0.100241	0.226680	
concave_points_se	...	0.358127	0.086741	0.394999	
symmetry_se	...	-0.128121	-0.077473	-0.103753	
fractal_dimension_se	...	-0.037488	-0.003195	-0.001000	
radius_worst	...	1.000000	0.359921	0.993708	
texture_worst	...	0.359921	1.000000	0.365098	
perimeter_worst	...	0.993708	0.365098	1.000000	
area_worst	...	0.984015	0.345842	0.977578	
smoothness_worst	...	0.216574	0.225429	0.236775	
compactness_worst	...	0.475820	0.360832	0.529408	
concavity_worst	...	0.573975	0.368366	0.618344	
concave_points_worst	...	0.787424	0.359755	0.816322	
symmetry_worst	...	0.243529	0.233027	0.269493	
fractal_dimension_worst	...	0.093492	0.219122	0.138957	

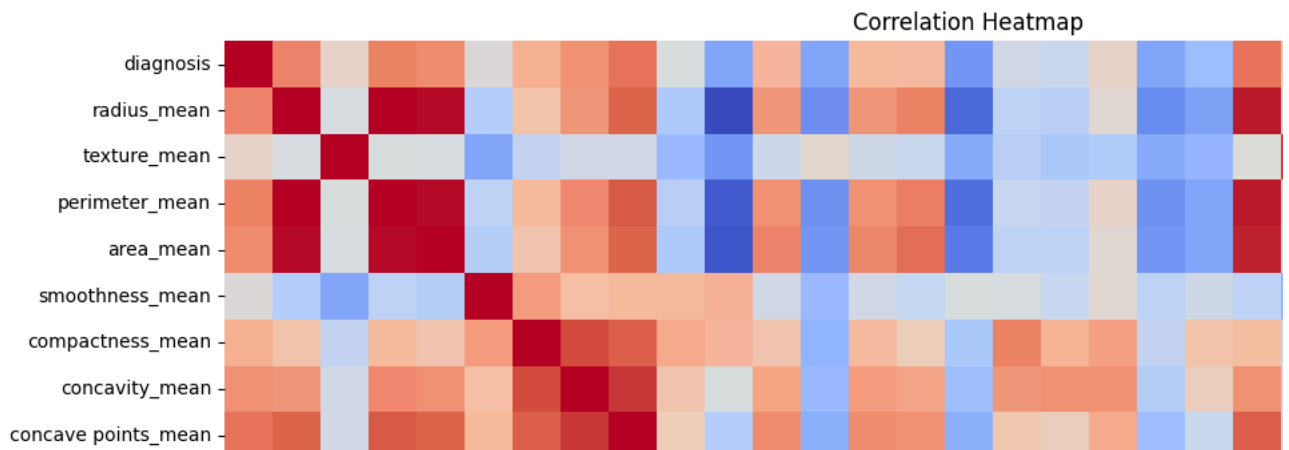
	area_worst	smoothness_worst	compactness_worst	\
diagnosis	0.733825	0.421465	0.590998	
radius_mean	0.941082	0.119616	0.413463	
texture_mean	0.343546	0.077503	0.277830	
perimeter_mean	0.941550	0.150549	0.455774	
area_mean	0.959213	0.123523	0.390410	
smoothness_mean	0.206718	0.805324	0.472468	
compactness_mean	0.509604	0.565541	0.865809	
concavity_mean	0.675987	0.448822	0.754968	
concave_points_mean	0.809630	0.452753	0.667454	
symmetry_mean	0.177193	0.426675	0.473200	
fractal_dimension_mean	-0.231854	0.504942	0.458798	
radius_se	0.751548	0.141919	0.287103	
texture_se	-0.083195	-0.073658	-0.092439	
perimeter_se	0.730713	0.130054	0.341919	
area_se	0.811408	0.125389	0.283257	
smoothness_se	-0.182195	0.314457	-0.055558	
compactness_se	0.199371	0.227394	0.678780	
concavity_se	0.188353	0.168481	0.484858	
concave_points_se	0.342271	0.215351	0.452888	
symmetry_se	-0.110343	-0.012662	0.060255	
fractal_dimension_se	-0.022736	0.170568	0.390159	
radius_worst	0.984015	0.216574	0.475820	
texture_worst	0.345842	0.225429	0.360832	
perimeter_worst	0.977578	0.236775	0.529408	
area_worst	1.000000	0.209145	0.438296	
smoothness_worst	0.209145	1.000000	0.568187	
compactness_worst	0.438296	0.568187	1.000000	
concavity_worst	0.543331	0.518523	0.892261	
concave_points_worst	0.747419	0.547691	0.801080	
symmetry_worst	0.209146	0.493838	0.614441	
fractal_dimension_worst	0.079647	0.617624	0.810455	

	concavity_worst	concave_points_worst	\
diagnosis	0.659610	0.793566	

radius_mean	0.526911	0.744214
texture_mean	0.301025	0.295316
perimeter_mean	0.563879	0.771241
area_mean	0.512606	0.722017
smoothness_mean	0.434926	0.503053
compactness_mean	0.816275	0.815573
concavity_mean	0.884103	0.861323
concave points_mean	0.752399	0.910155
symmetry_mean	0.433721	0.430297
fractal_dimension_mean	0.346234	0.175325
radius_se	0.380585	0.531062
texture_se	-0.068956	-0.119638
perimeter_se	0.418899	0.554897
area_se	0.385100	0.538166
smoothness_se	-0.058298	-0.102007
compactness_se	0.639147	0.483208
concavity_se	0.662564	0.440472
concave points_se	0.549592	0.602450
symmetry_se	0.037119	-0.030413
fractal_dimension_se	0.379975	0.215204
radius_worst	0.573975	0.787424
texture_worst	0.368366	0.359755
perimeter_worst	0.618344	0.816322
area_worst	0.543331	0.747419
smoothness_worst	0.518523	0.547691
compactness_worst	0.892261	0.801080
concavity_worst	1.000000	0.855434
concave points_worst	0.855434	1.000000
symmetry_worst	0.532520	0.502528
fractal_dimension_worst	0.686511	0.511114

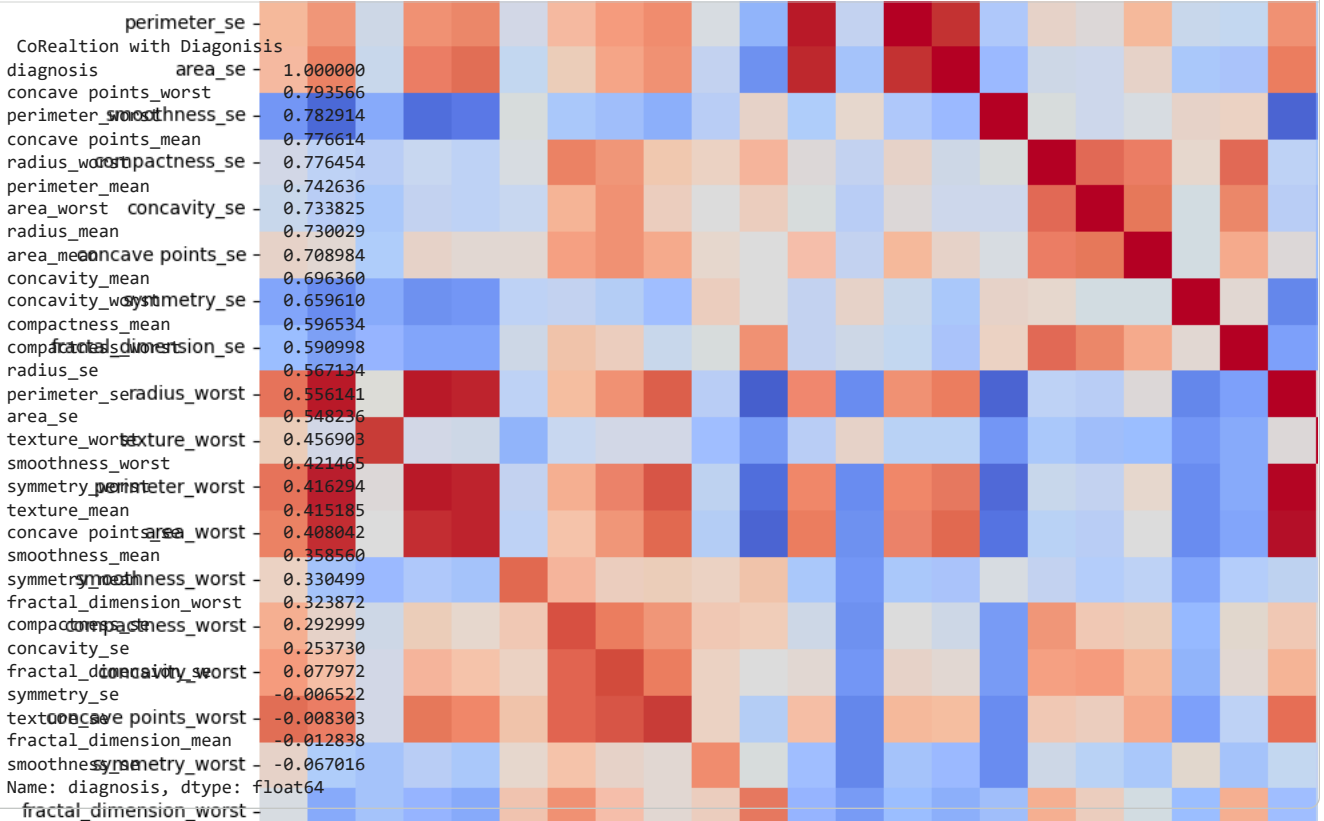
	symmetry_worst	fractal_dimension_worst
diagnosis	0.416294	0.323872
radius_mean	0.163953	0.007066
texture_mean	0.105008	0.119205
perimeter_mean	0.189115	0.051019
area_mean	0.143570	0.003738
smoothness_mean	0.394309	0.499316
compactness_mean	0.510223	0.687382
concavity_mean	0.409464	0.514930
concave points_mean	0.375744	0.368661
symmetry_mean	0.699826	0.438413
fractal_dimension_mean	0.334019	0.767297
radius_se	0.094543	0.049559
texture_se	-0.128215	-0.045655
perimeter_se	0.109930	0.085433
area_se	0.074126	0.017539
smoothness_se	-0.107342	0.101480
compactness_se	0.277878	0.590973
concavity_se	0.197788	0.439329
concave points_se	0.143116	0.310655
symmetry_se	0.389402	0.078079
fractal_dimension_se	0.111094	0.591328
radius_worst	0.243529	0.093492
texture_worst	0.233027	0.219122
perimeter_worst	0.269493	0.138957
area_worst	0.209146	0.079647
smoothness_worst	0.493838	0.617624
compactness_worst	0.614441	0.810455
concavity_worst	0.532520	0.686511
concave points_worst	0.502528	0.511114
symmetry_worst	1.000000	0.537848
fractal_dimension_worst	0.537848	1.000000

[31 rows x 31 columns]



Corelation With Target

```
target_correlation = correlation_matrix["diagnosis"].sort_values(ascending=False)
print("\n CoRealtion with Diagonis")
print(target_correlation)
```



5. Covariance Analysis

```
covariance_matrix = df.cov()

print("\nCovariance Matrix")
print(covariance_matrix)

plt.figure(figsize=(18, 14))
sns.heatmap(covariance_matrix, cmap="coolwarm", annot=False)

plt.title("Covariance Heatmap")
plt.show()
```


Covariance Matrix

	diagnosis	radius_mean	texture_mean	\
diagnosis	0.234177	1.244954	0.864145	
radius_mean	1.244954	12.418920	4.907582	
texture_mean	0.864145	4.907582	18.498909	
perimeter_mean	8.732438	85.447142	34.439759	
area_mean	120.738222	1224.483409	485.993787	
smoothness_mean	0.002440	0.008454	-0.001415	
compactness_mean	0.015246	0.094197	0.053767	
concavity_mean	0.026864	0.190128	0.103692	
concave points_mean	0.014583	0.112475	0.048977	
symmetry_mean	0.004384	0.014273	0.008419	
fractal_dimension_mean	-0.000044	-0.007754	-0.002321	
radius_se	0.076107	0.663650	0.329037	
texture_se	-0.002217	-0.189189	0.916695	
perimeter_se	0.544135	4.803550	2.449449	
area_se	12.068819	117.968162	50.840865	
smoothness_se	-0.000097	-0.002355	0.000085	
compactness_se	0.002539	0.013001	0.014787	
concavity_se	0.003706	0.020659	0.018604	
concave points_se	0.001218	0.008180	0.004348	
symmetry_se	-0.000026	-0.003039	0.000325	
fractal_dimension_se	0.000100	-0.000398	0.000620	
radius_worst	1.816042	16.513749	7.329267	
texture_worst	1.358959	6.433100	24.110148	
perimeter_worst	12.730868	114.288570	51.745933	
area_worst	202.184994	1888.227223	841.283832	
smoothness_worst	0.004657	0.009625	0.007611	
compactness_worst	0.044997	0.229249	0.188010	
concavity_worst	0.066592	0.387386	0.270110	
concave points_worst	0.025243	0.172393	0.083491	
symmetry_worst	0.012463	0.035746	0.027942	
fractal_dimension_worst	0.002831	0.000450	0.009260	

	perimeter_mean	area_mean	smoothness_mean	\
diagnosis	8.732438	120.738222	0.002440	
radius_mean	85.447142	1224.483409	0.008454	
texture_mean	34.439759	485.993787	-0.001415	
perimeter_mean	590.440480	8435.772345	0.070836	
area_mean	8435.772345	123843.554318	0.876178	
smoothness_mean	0.070836	0.876178	0.000198	
compactness_mean	0.714714	9.264931	0.000490	
concavity_mean	1.387234	19.244924	0.000585	
concave points_mean	0.802360	11.241958	0.000302	
symmetry_mean	0.121922	1.459596	0.000215	
fractal_dimension_mean	-0.044859	-0.703426	0.000058	
radius_se	4.661401	71.490945	0.001176	
texture_se	-1.162988	-12.867168	0.000531	
perimeter_se	34.053028	517.009995	0.008420	
area_se	823.492755	12808.517580	0.157742	
smoothness_se	-0.014788	-0.176221	0.000014	
compactness_se	0.109111	1.339725	0.000080	
concavity_se	0.167296	2.205952	0.000105	
concave points_se	0.061055	0.808460	0.000033	
symmetry_se	-0.016396	-0.210896	0.000023	
fractal_dimension_se	-0.000355	-0.018519	0.000011	
radius_worst	113.858063	1637.521341	0.014487	
texture_worst	45.258113	621.824934	0.003118	
perimeter_worst	792.328208	11341.789807	0.112879	
area_worst	13026.148359	192192.557633	1.655299	
smoothness_worst	0.083526	0.992514	0.000259	
compactness_worst	1.742478	21.616602	0.001045	
concavity_worst	2.858506	37.634415	0.001276	
concave points_worst	1.231848	16.701789	0.000465	
symmetry_worst	0.284300	3.125809	0.000343	
fractal_dimension_worst	0.022391	0.023756	0.000127	

	compactness_mean	concavity_mean	\
diagnosis	0.015246	0.026864	
radius_mean	0.094197	0.190128	
texture_mean	0.053767	0.103692	
perimeter_mean	0.714714	1.387234	
area_mean	9.264931	19.244924	
smoothness_mean	0.000490	0.000585	
compactness_mean	0.002789	0.003718	
concavity_mean	0.003718	0.006355	
concave points_mean	0.001703	0.002850	
symmetry_mean	0.000873	0.001094	
fractal_dimension_mean	0.000211	0.000190	
radius_se	0.007286	0.013970	
texture_se	0.001346	0.003352	
perimeter_se	0.058612	0.106443	
area_se	1.094708	2.239119	

smoothness_se	0.000021	0.000024
compactness_se	0.000699	0.000957
concavity_se	0.000910	0.001663
concave points_se	0.000209	0.000336
symmetry_se	0.000100	0.000117
fractal_dimension_se	0.000071	0.000095
radius_worst	0.136643	0.265181
texture_worst	0.080544	0.146934
perimeter_worst	1.047413	1.954350
area_worst	15.323436	30.682405
smoothness_worst	0.000682	0.000817
compactness_worst	0.007194	0.009469
concavity_worst	0.008994	0.014704
concave points_worst	0.002831	0.004513
symmetry_worst	0.001667	0.002020
fractal_dimension_worst	0.000656	0.000741

	concave points_mean	symmetry_mean	...	\
diagnosis	0.014583	0.004384	...	
radius_mean	0.112475	0.014273	...	
texture_mean	0.048977	0.008419	...	
perimeter_mean	0.802360	0.121922	...	
area_mean	11.241958	1.459596	...	
smoothness_mean	0.000302	0.000215	...	
compactness_mean	0.001703	0.000873	...	
concavity_mean	0.002850	0.001094	...	
concave points_mean	0.001506	0.000492	...	
symmetry_mean	0.000492	0.000752	...	
fractal_dimension_mean	0.000046	0.000093	...	
radius_se	0.007511	0.002306	...	
texture_se	0.000460	0.001937	...	
perimeter_se	0.055753	0.017398	...	
area_se	1.218501	0.279314	...	
smoothness_se	0.000003	0.000015	...	
compactness_se	0.000341	0.000207	...	
concavity_se	0.000514	0.000284	...	
concave points_se	0.000147	0.000067	...	
symmetry_se	0.000031	0.000102	...	
fractal_dimension_se	0.000026	0.000024	...	
radius_worst	0.155721	0.024609	...	
texture_worst	0.069819	0.015274	...	
perimeter_worst	1.116016	0.201896	...	
area_worst	17.886881	2.765725	...	
smoothness_worst	0.000401	0.000267	...	
compactness_worst	0.004075	0.002041	...	
concavity_worst	0.006091	0.002481	...	
concave points_worst	0.002321	0.000775	...	
symmetry_worst	0.000902	0.001187	...	
fractal_dimension_worst	0.000258	0.000217	...	

	radius_worst	texture_worst	perimeter_worst	\
diagnosis	1.816042	1.358959	12.730868	
radius_mean	16.513749	6.433100	114.288570	
texture_mean	7.329267	24.110148	51.745933	
perimeter_mean	113.858063	45.258113	792.328208	
area_mean	1637.521341	621.824934	11341.789807	
smoothness_mean	0.014487	0.003118	0.112879	
compactness_mean	0.136643	0.080544	1.047413	
concavity_mean	0.265181	0.146934	1.954350	
concave points_mean	0.155721	0.069819	1.116016	
symmetry_mean	0.024609	0.015274	0.201896	
fractal_dimension_mean	-0.008657	-0.002225	-0.048671	
radius_se	0.958416	0.332022	6.706311	
texture_se	-0.297794	1.386754	-1.895237	
perimeter_se	6.813122	2.489976	48.986473	
area_se	166.522900	54.940353	1163.599863	
smoothness_se	-0.003348	-0.001379	-0.021924	
compactness_se	0.017710	0.015740	0.156768	
concavity_se	0.027269	0.018598	0.229928	
concave points_se	0.010680	0.003290	0.081898	
symmetry_se	-0.005119	-0.003936	-0.028820	
fractal_dimension_se	-0.000479	-0.000052	-0.000089	
radius_worst	23.360224	10.691931	161.387312	
texture_worst	10.691931	37.776483	75.403697	
perimeter_worst	161.387312	75.403697	1129.130847	
area_worst	2707.850532	1210.245587	18702.869991	
smoothness_worst	0.023900	0.031635	0.181660	
compactness_worst	0.361835	0.348936	2.798929	
concavity_worst	0.578757	0.472340	4.334781	
concave points_worst	0.250165	0.145344	1.803071	
symmetry_worst	0.072820	0.088609	0.560250	
fractal_dimension_worst	0.008161	0.024325	0.084334	

	area_worst	smoothness_worst	compactness_worst	\
diagnosis	202.184994	0.004657	0.044997	

radius_mean	1888.227223	0.009625	0.229249
texture_mean	841.283832	0.007611	0.188010
perimeter_mean	13026.148359	0.083526	1.742478
area_mean	192192.557633	0.992514	21.616602
smoothness_mean	1.655299	0.000259	0.001045
compactness_mean	15.323436	0.000682	0.007194
concavity_mean	30.682405	0.000817	0.009469
concave points_mean	17.886881	0.000401	0.004075
symmetry_mean	2.765725	0.000267	0.002041
fractal_dimension_mean	-0.932024	0.000081	0.000510
radius_se	118.661947	0.000899	0.012527
texture_se	-26.130287	-0.000928	-0.008023
perimeter_se	841.165375	0.006004	0.108769
area_se	21015.970960	0.130238	2.027379
smoothness_se	-0.311464	0.000022	-0.000026
compactness_se	2.032819	0.000093	0.001913
concavity_se	3.237150	0.000116	0.002303
concave points_se	1.202431	0.000030	0.000440
symmetry_se	-0.519330	-0.000002	0.000078
fractal_dimension_se	-0.034253	0.000010	0.000162
radius_worst	2707.850532	0.023900	0.361835
texture_worst	1210.245587	0.031635	0.348936
perimeter_worst	18702.869991	0.181660	2.798929
area_worst	324167.385102	2.718848	39.262857
smoothness_worst	2.718848	0.000521	0.002041
compactness_worst	39.262857	0.002041	0.024755
concavity_worst	64.537720	0.002470	0.029288
concave points_worst	27.972274	0.000822	0.008285
symmetry_worst	7.367083	0.000698	0.005981
fractal_dimension_worst	0.819035	0.000255	0.002303

	concavity_worst	concave points_worst \
diagnosis	0.066592	0.025243
radius_mean	0.387386	0.172393
texture_mean	0.270110	0.083491
perimeter_mean	2.858506	1.231848
area_mean	37.634415	16.701789
smoothness_mean	0.001276	0.000465
compactness_mean	0.008994	0.002831
concavity_mean	0.014704	0.004513
concave points_mean	0.006091	0.002321
symmetry_mean	0.002481	0.000775
fractal_dimension_mean	0.000510	0.000081
radius_se	0.022018	0.009680
texture_se	-0.007936	-0.004338
perimeter_se	0.176695	0.073747
area_se	3.654804	1.609241
smoothness_se	-0.000037	-0.000020
compactness_se	0.002388	0.000569
concavity_se	0.004173	0.000874
concave points_se	0.000707	0.000244
symmetry_se	0.000064	-0.000017
fractal_dimension_se	0.000210	0.000037
radius_worst	0.578757	0.250165
texture_worst	0.472340	0.145344
perimeter_worst	4.334781	1.803071
area_worst	64.537720	27.972274
smoothness_worst	0.002470	0.000822
compactness_worst	0.029288	0.008285
concavity_worst	0.043524	0.011731
concave points_worst	0.011731	0.004321
symmetry_worst	0.006873	0.002044
fractal_dimension_worst	0.002587	0.000607

	symmetry_worst	fractal_dimension_worst
diagnosis	0.012463	0.002831
radius_mean	0.035746	0.000450
texture_mean	0.027942	0.009260
perimeter_mean	0.284300	0.022391
area_mean	3.125809	0.023756
smoothness_mean	0.000343	0.000127
compactness_mean	0.001667	0.000656
concavity_mean	0.002020	0.000741
concave points_mean	0.000902	0.000258
symmetry_mean	0.001187	0.000217
fractal_dimension_mean	0.000146	0.000098
radius_se	0.001622	0.000248
texture_se	-0.004376	-0.000455
perimeter_se	0.013751	0.003120
area_se	0.208622	0.014411
smoothness_se	-0.000020	0.000006
compactness_se	0.000308	0.000191
concavity_se	0.000369	0.000240
concave points_se	0.000055	0.000035
symmetry_se	0.000199	0.000012

6. Train-Test-Split

	0.00018	0.00028
fractal_dimension_se	0.072820	0.008161
radius_worst	0.088609	0.024325
texture_worst	0.560350	0.084324

```
# =====
# STEP 30: SELECT USEFUL FEATURES FOR CLASSIFICATION
# =====
```

```
selected_features = [
    "radius_mean",
    "perimeter_mean",
    "area_mean",
    "concavity_mean",
    "concave points_mean",
    "radius_worst",
    "perimeter_worst",
    "area_worst",
    "concavity_worst",
    "concave points_worst"
]
```

```
X = df[selected_features]
y = df["diagnosis"]
```

```
print("Selected features:")
print(selected_features)
```

```
print("\nFeature shape:", X.shape)
print("Target shape:", y.shape)
```

```
Selected features:
['radius_mean', 'perimeter_mean', 'area_mean', 'concavity_mean', 'concave points_mean', 'radius_worst', 'perimeter_worst', 'area_worst', 'concavity_worst', 'concave points_worst']
Feature shape: (569, 10)
Target shape: (569,)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

```
print("\nTrain-Test-split")
```

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
fractal_dimension_worst
Train-Test-split
Training data: (455, 10)
Testing data: (114, 10)
```

```
# Standardization
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("\nData standardization completed.")
```

Data standardization completed.

7. Logistic Regression

```
logistic_model = LogisticRegression(max_iter=2000, random_state=42)

logistic_model.fit(X_train_scaled, y_train)

logistic_prediction = logistic_model.predict(X_test_scaled)

logistic_accuracy = accuracy_score(y_test, logistic_prediction)

logistic_precision = precision_score(y_test, logistic_prediction)
```

```

logistic_recall = recall_score(
    y_test,
    logistic_prediction
)

logistic_f1 = f1_score(
    y_test,
    logistic_prediction
)

print("\n LOGISTIC REGRESSION ")

print("Accuracy:", logistic_accuracy)
print("Precision:", logistic_precision)
print("Recall:", logistic_recall)
print("F1 Score:", logistic_f1)

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        logistic_prediction,
        target_names=["Benign", "Malignant"]
    )
)

```

```

LOGISTIC REGRESSION
Accuracy: 0.956140350877193
Precision: 0.9743589743589743
Recall: 0.9047619047619048
F1 Score: 0.9382716049382716

Classification Report:

```

	precision	recall	f1-score	support
Benign	0.95	0.99	0.97	72
Malignant	0.97	0.90	0.94	42
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

8. Decision Tree Classifier

```

decision_tree = DecisionTreeClassifier(random_state=42)

decision_tree.fit(X_train,y_train)

dt_prediction = decision_tree.predict(X_test)

dt_accuracy = accuracy_score(y_test,dt_prediction)

dt_precision = precision_score(y_test,dt_prediction)

dt_recall = recall_score(
    y_test,
    dt_prediction
)

dt_f1 = f1_score(
    y_test,
    dt_prediction
)

print("\nDecision Tree")

print("Accuracy:", dt_accuracy)
print("Precision:", dt_precision)
print("Recall:", dt_recall)
print("F1 Score:", dt_f1)

```

```
Decision Tree
Accuracy: 0.9035087719298246
Precision: 0.918918918918919
Recall: 0.8095238095238095
F1 Score: 0.8607594936708861
```

9. Random Forest Classifier

```
random_forest = RandomForestClassifier(n_estimators=200,random_state=42)

random_forest.fit(X_train,y_train)

rf_prediction = random_forest.predict( X_test)

rf_accuracy = accuracy_score(y_test,rf_prediction)

rf_precision = precision_score(
    y_test,
    rf_prediction
)

rf_recall = recall_score(
    y_test,
    rf_prediction
)

rf_f1 = f1_score(
    y_test,
    rf_prediction
)

print("\nRandom Forest")

print("Accuracy:", rf_accuracy)
print("Precision:", rf_precision)
print("Recall:", rf_recall)
print("F1 Score:", rf_f1)
```

```
Random Forest
Accuracy: 0.956140350877193
Precision: 1.0
Recall: 0.8809523809523809
F1 Score: 0.9367088607594937
```

10. K-Nearest Neighbors — KNN

```
knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train_scaled,y_train)

knn_prediction = knn.predict(X_test_scaled)

knn_accuracy = accuracy_score(y_test, knn_prediction)

knn_precision = precision_score( y_test,knn_prediction)

knn_recall = recall_score(
    y_test,
    knn_prediction
)

knn_f1 = f1_score(
    y_test,
    knn_prediction
)

print("\nKNN")

print("Accuracy:", knn_accuracy)
print("Precision:", knn_precision)
print("Recall:", knn_recall)
```

```
print("F1 Score:", knn_f1)
```

KNN

```
Accuracy: 0.956140350877193
Precision: 1.0
Recall: 0.8809523809523809
F1 Score: 0.9367088607594937
```

11. Support Vector Machine — SVM

```
svm = SVC(kernel="rbf", random_state=42)

svm.fit(X_train_scaled, y_train)

svm_prediction = svm.predict(X_test_scaled)

svm_accuracy = accuracy_score(
    y_test,
    svm_prediction
)

svm_precision = precision_score(
    y_test,
    svm_prediction
)

svm_recall = recall_score(
    y_test,
    svm_prediction
)

svm_f1 = f1_score(
    y_test,
    svm_prediction
)

print("\nSupport Vector Machine")

print("Accuracy:", svm_accuracy)
print("Precision:", svm_precision)
print("Recall:", svm_recall)
print("F1 Score:", svm_f1)
```

```
Support Vector Machine
Accuracy: 0.956140350877193
Precision: 1.0
Recall: 0.8809523809523809
F1 Score: 0.9367088607594937
```

```
# Confusion Matrix
models_predictions = {
    "Logistic Regression": logistic_prediction,
    "Decision Tree": dt_prediction,
    "Random Forest": rf_prediction,
    "KNN": knn_prediction,
    "SVM": svm_prediction
}

for model_name, prediction in models_predictions.items():

    cm = confusion_matrix(
        y_test,
        prediction
    )

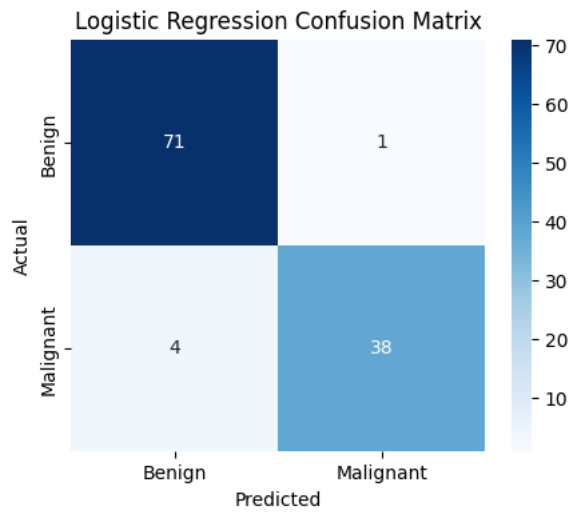
    plt.figure(figsize=(5, 4))

    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
```

```
        cmap="Blues",
        xticklabels=["Benign", "Malignant"],
        yticklabels=["Benign", "Malignant"]
    )

    plt.title(model_name + " Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")

    plt.show()
```

```
# Model Comparison
results = pd.DataFrame({

    "Model": [
        "Logistic Regression",
        "Decision Tree",
        "Random Forest",
        "KNN",
        "SVM"
    ],

    "Accuracy": [
        logistic_accuracy,
        dt_accuracy,
        rf_accuracy,
        knn_accuracy,
        svm_accuracy
    ],

    "Precision": [
        logistic_precision,
        dt_precision,
        rf_precision,
        knn_precision,
        svm_precision
    ],

    "Recall": [
        logistic_recall,
        dt_recall,
        rf_recall,
        knn_recall,
        svm_recall
    ],

    "F1 Score": [
        logistic_f1,
        dt_f1,
        rf_f1,
        knn_f1,
        svm_f1
    ]
})

print("\nModel Comparison")

print(
    results.sort_values(
        by="Accuracy",
```

```

        ascending=False
    )
)

```

SVM Confusion Matrix

Model	Comaprision	Model	Accuracy	Precision	Recall	F1 Score
0	Benign	Logistic Regression	0.956140	0.974359	0.904762	0.938272
2		Random Forest	0.956140	1.000000	0.880952	0.936709
3		KNN	0.956140	1.000000	0.880952	0.936709
4		SVM	0.956140	1.000000	0.880952	0.936709
1	Malignant	Decision Tree	0.903509	0.918919	0.809524	0.860759

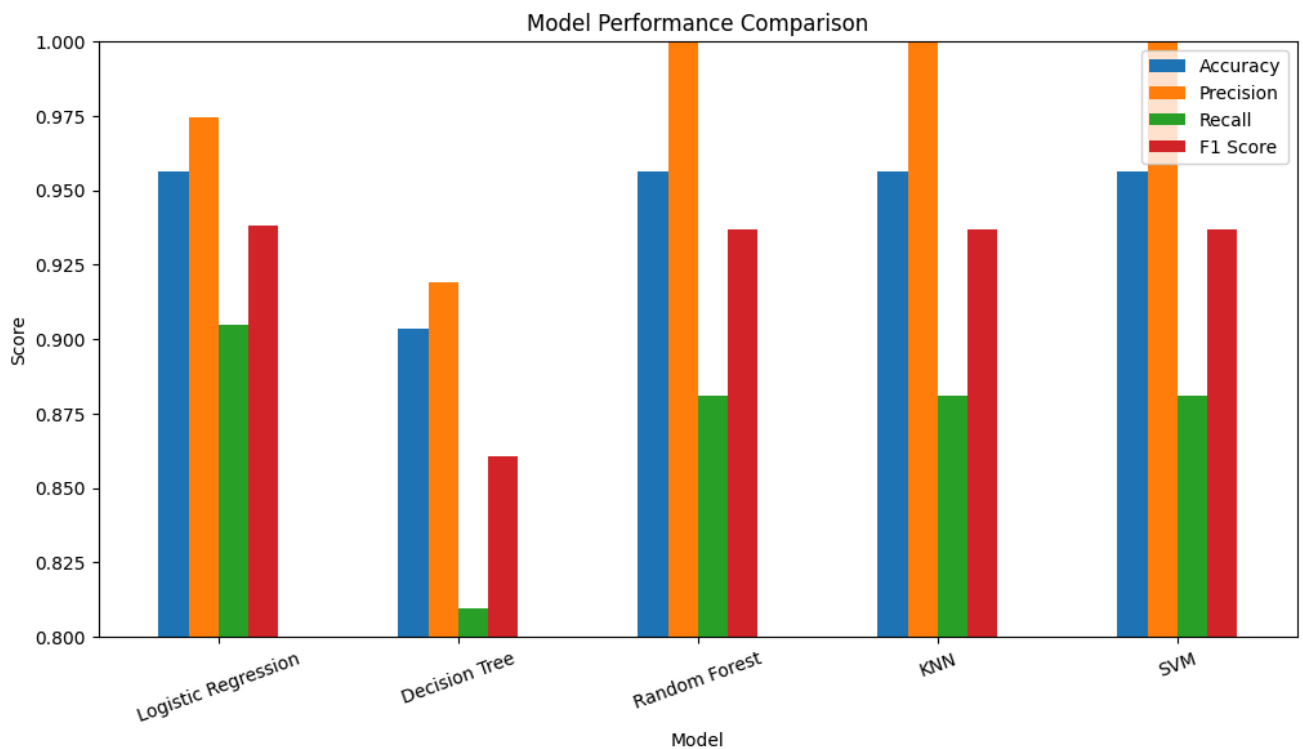
```

# Model Comaprision Graph
results_plot = results.set_index("Model")

results_plot.plot(kind="bar",figsize=(12, 6))

plt.title("Model Performance Comparison")
plt.ylabel("Score")
plt.ylim(0.8, 1.0)
plt.xticks(rotation=20)
plt.show()

```



12. Model Selection

```

best_model_name = results.loc[
    results["Accuracy"].idxmax(),
    "Model"
]

print("\nBest Model")
print("Best Model:", best_model_name)

# Select corresponding model and scaler requirement

if best_model_name == "Logistic Regression":
    best_model = logistic_model
    best_scaler = scaler
    scaling_required = True

```

```

elif best_model_name == "Decision Tree":
    best_model = decision_tree
    best_scaler = None
    scaling_required = False

elif best_model_name == "Random Forest":
    best_model = random_forest
    best_scaler = None
    scaling_required = False

elif best_model_name == "KNN":
    best_model = knn
    best_scaler = scaler
    scaling_required = True

elif best_model_name == "SVM":
    best_model = svm
    best_scaler = scaler
    scaling_required = True

```

Best Model
Best Model: Logistic Regression

13. Save the Best Model using Joblib

```

joblib.dump(
    best_model,
    "breast_cancer_best_model.pkl"
)

print("\nBest model saved as:")
print("breast_cancer_best_model.pkl")

# Save scaler if required
if scaling_required:

    joblib.dump(
        best_scaler,
        "breast_cancer_scaler.pkl"
    )

    print("Scaler saved as:")
    print("breast_cancer_scaler.pkl")

# Save feature names
joblib.dump(
    X.columns.tolist(),
    "breast_cancer_features.pkl"
)

print("Feature names saved as:")
print("breast_cancer_features.pkl")

```

Best model saved as:
breast_cancer_best_model.pkl
Scaler saved as:
breast_cancer_scaler.pkl
Feature names saved as:
breast_cancer_features.pkl

```

# Test Saved Model
loaded_model = joblib.load(
    "breast_cancer_best_model.pkl"
)

if scaling_required:

    loaded_scaler = joblib.load(

```

```
        "breast_cancer_scaler.pkl"
    )

    X_test_for_prediction = loaded_scaler.transform(
        X_test
    )

else:

    X_test_for_prediction = X_test

saved_prediction = loaded_model.predict(
    X_test_for_prediction
)

saved_accuracy = accuracy_score(
    y_test,
    saved_prediction
)
```

Accuracy of saved model: 0.956140350877193

```
# Sample Prediction
sample = X.iloc[[0]]

if scaling_required:

    sample_scaled = loaded_scaler.transform(
        sample
    )

    sample_prediction = loaded_model.predict(
        sample_scaled
    )

else:

    sample_prediction = loaded_model.predict(
```